

RESTful Schema-Aware Data Integration & Transformation API

RESTful Schema-Aware Data Integration & Transformation API

A Final Project Presentation By:

1. Aaron Paul
2. Athuljith S
3. Dijith Dinesh
4. Peter Joe Menachery
5. Pranav Manoj

Introduction: Project Domain

- **Domains**
 - Backend Services
 - Data Integration
 - RESTful APIs
- It provides a foundational middleware solution that can be used by any application needing to expose database information securely and efficiently over the web.

Introduction: Problem Statement

- In modern software, client applications are often **tightly coupled** to the backend database schema.
- This means a minor change in the database (like renaming a column) can cause a **cascade of failures** in all connected applications.
- This "brittleness" leads to fragile systems, high maintenance costs, and significant developer inefficiency.

Introduction: Objectives

- To **decouple** client applications from the physical database structure.
- To **abstract** away the complexity of SQL with a simple, intuitive URL-based query language.
- To implement **on-the-fly data transformation** into multiple formats (JSON, CSV, XML).
- To enforce **robust, role-based security** using API keys.

Introduction: Technologies Used

- **Programming Language:** A pure Object-Oriented design in **Java (JDK 11+)**.
- **Core Libraries:** We intentionally used only standard Java libraries to build a lightweight, dependency-free application.
 - **Networking:** `com.sun.net.httpserver`
 - **Database:** `JDBC` (Java Database Connectivity)
- **Database: SQLite** for portability, with a design that allows easy migration to PostgreSQL or MySQL.

Proposed System: Overview of Our Solution

- We have built a **Universal Data Gateway**.
- It is a standalone, intelligent API server that acts as a middleware between any client application and a database.
- Clients communicate with our server using simple HTTP requests, and our server handles all the complex database interactions internally.

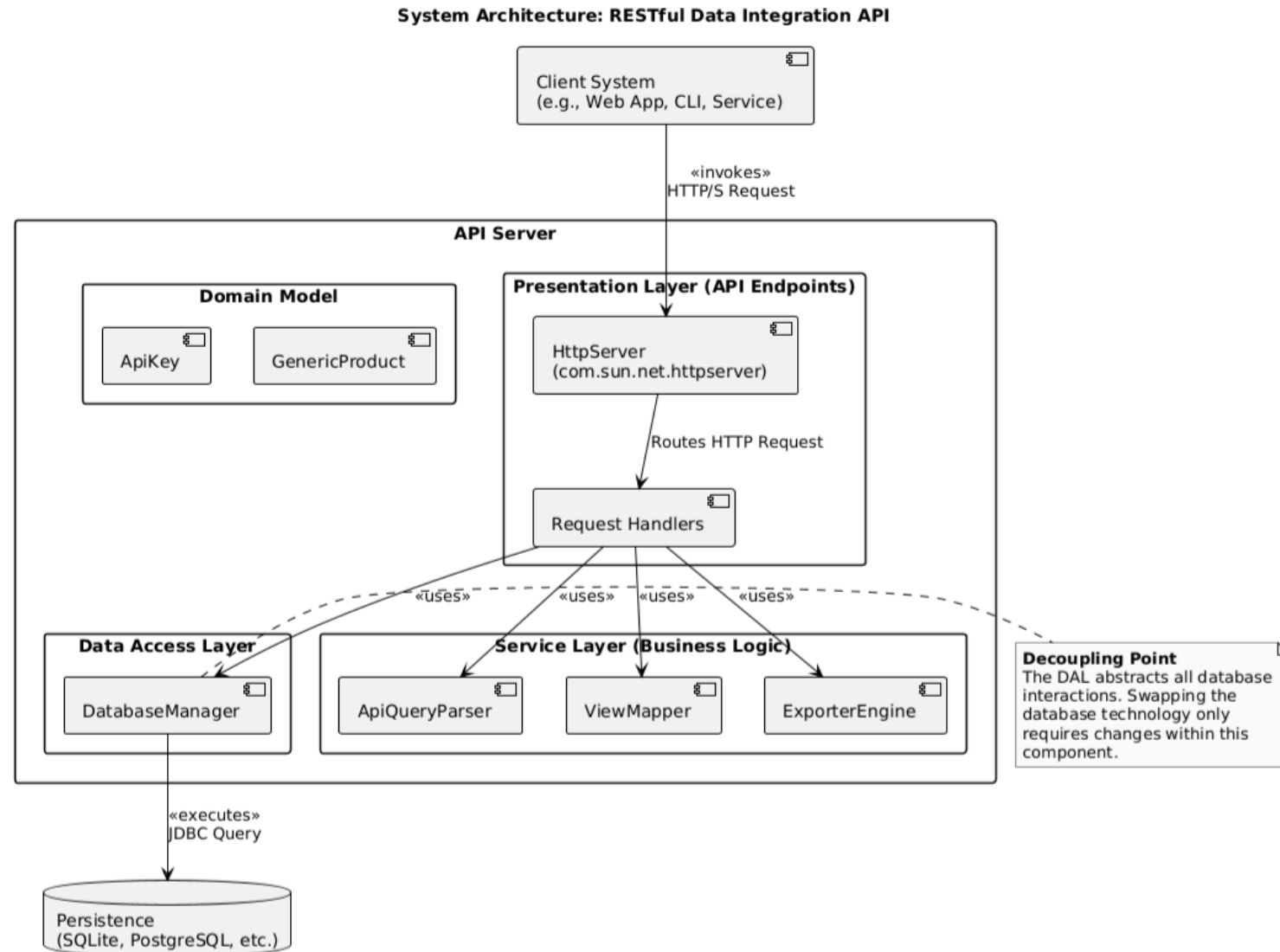
Proposed System: How It's Better

- **Decoupling:** Unlike traditional systems, if the database schema changes, only an API mapping needs to be updated. No client-side code has to be touched.
- **Efficiency:** It eliminates repetitive, error-prone data-handling code on the client side by centralizing all data logic on the server.
- **Flexibility:** Provides on-demand data transformation, allowing different clients to consume data in the exact format they need without any effort.

Proposed System: Key Features

- **Dynamic Query Engine:** Filter, sort, paginate, and select fields directly from the URL.
- **On-the-Fly Data Transformation:** Request data in JSON, CSV, XML, HTML, or Markdown.
- **Schema-Aware Views:** Dynamically rename data fields to match client-side models.
- **Role-Based Access Control (RBAC):** Secure endpoints with distinct read-only, write, and admin-level API keys.

Block Diagram / Architecture Diagram



Module Description

The project is organized into several distinct modules, each with a single, clear responsibility.

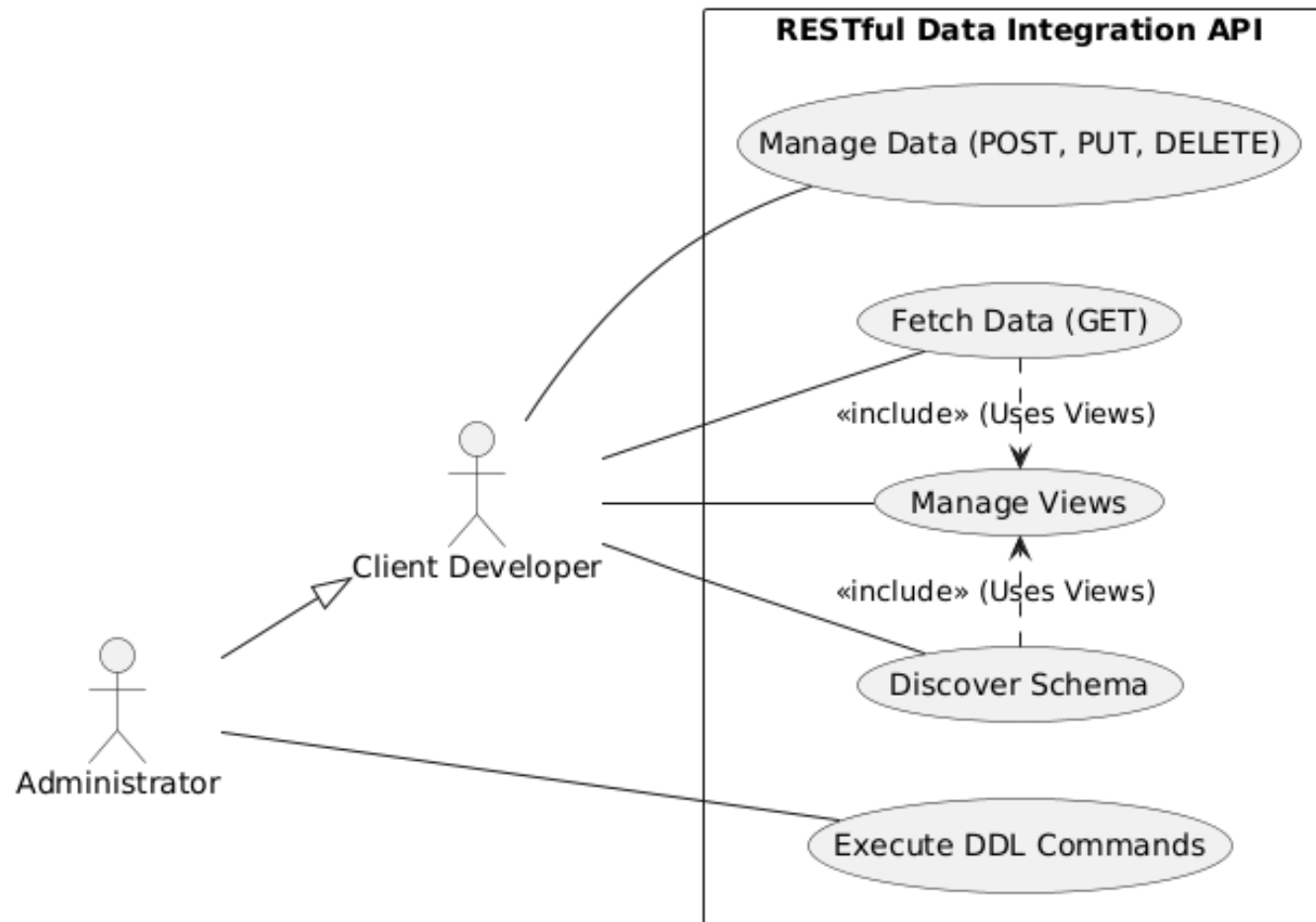
- `server` **Module:** The Presentation Layer. Handles all incoming HTTP traffic and routes requests.
- `util` **Module:** The "brains" of the application. Securely parses URLs into SQL and handles data translation.
- `db` **Module:** The Data Access Layer. The *only* module that communicates with the database, encapsulating all JDBC logic.
- `export` **Module:** The Transformation Engine. Formats data into JSON, CSV, etc., on demand.

Design Diagrams: Overview

- To ensure a robust and maintainable system, we used several industry-standard **UML and Data Flow diagrams** during the design phase.
- These diagrams act as the **blueprint** for our application's architecture and logic.
- The following slides will showcase the key diagrams we created.

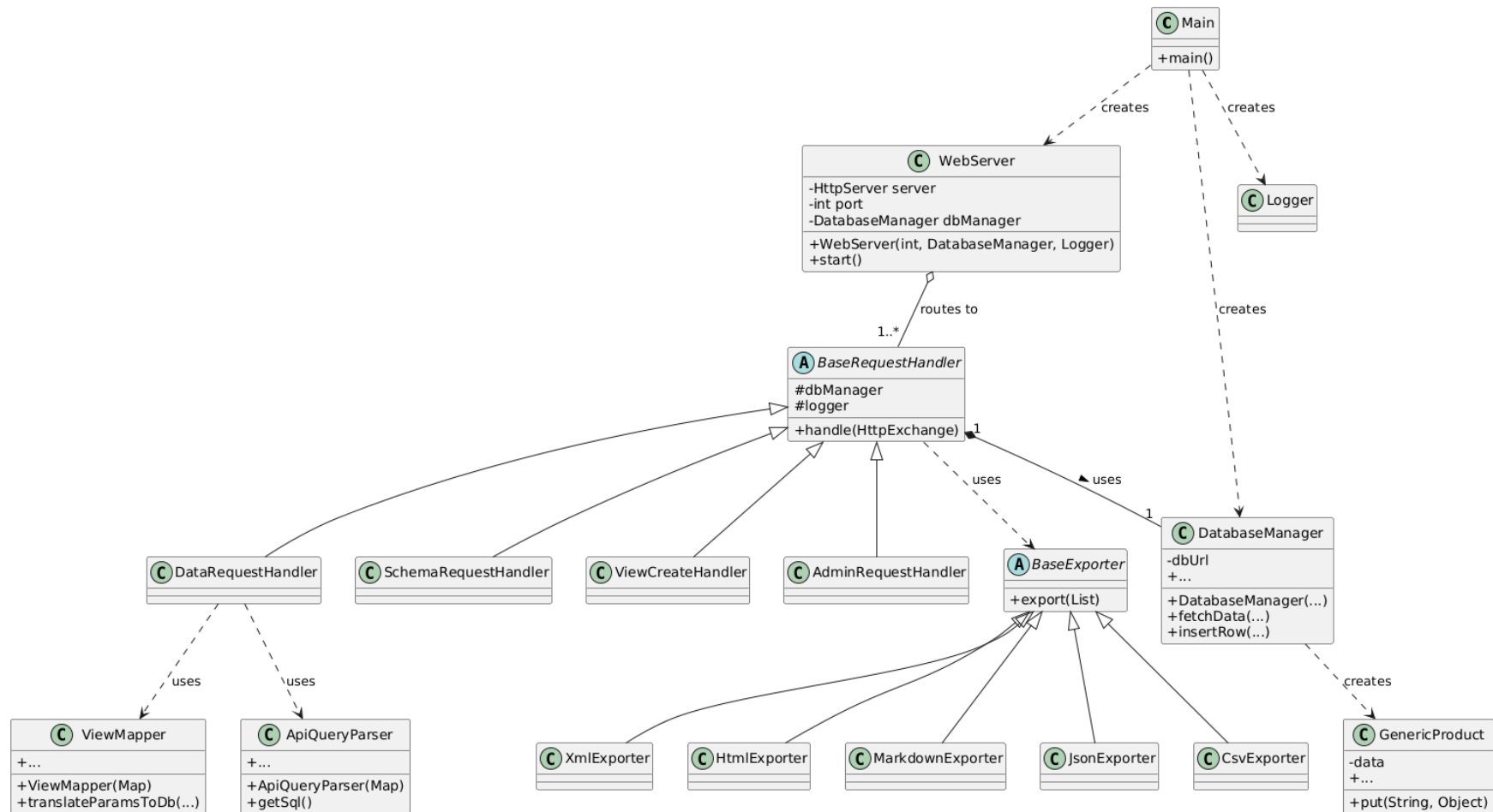
Design: Use Case Diagram

- This diagram shows how actors (Client Developer, Administrator) interact with the system's core functionalities.



Design: Class Diagram

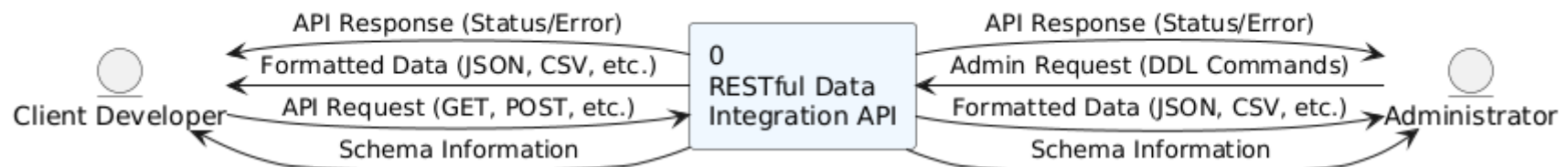
- This is the essential blueprint of our OOP design, detailing all classes, their attributes, methods, and the relationships between them.



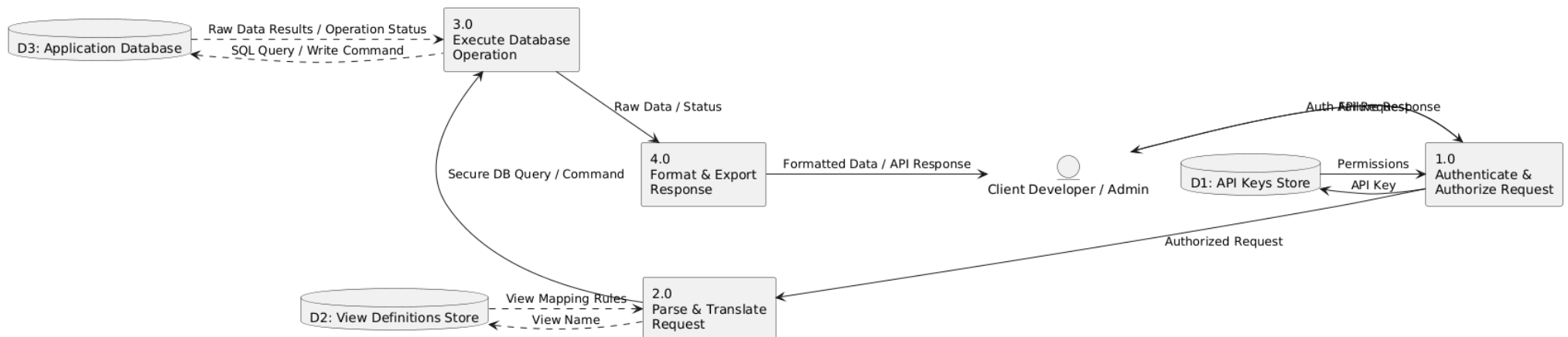
Design: Data Flow Diagrams (DFD)

- **DFD Level 0:** Shows the entire system as a single process.
- **DFD Level 1:** Details the major internal processes and data stores.

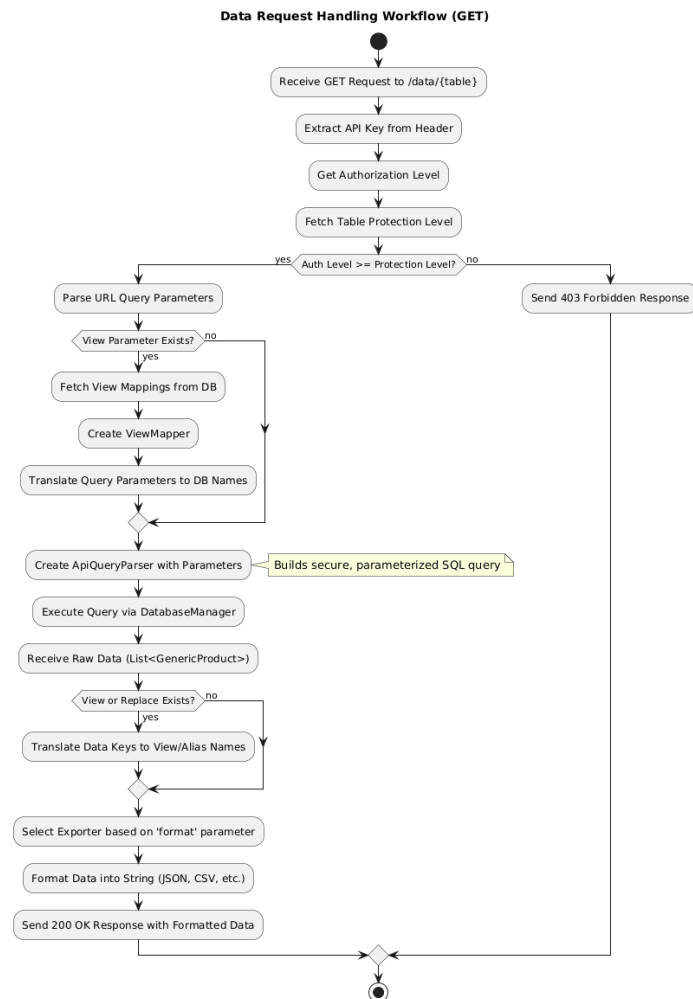
Data Flow Diagram (Level 0 - Context Diagram)



Data Flow Diagram (Level 1)

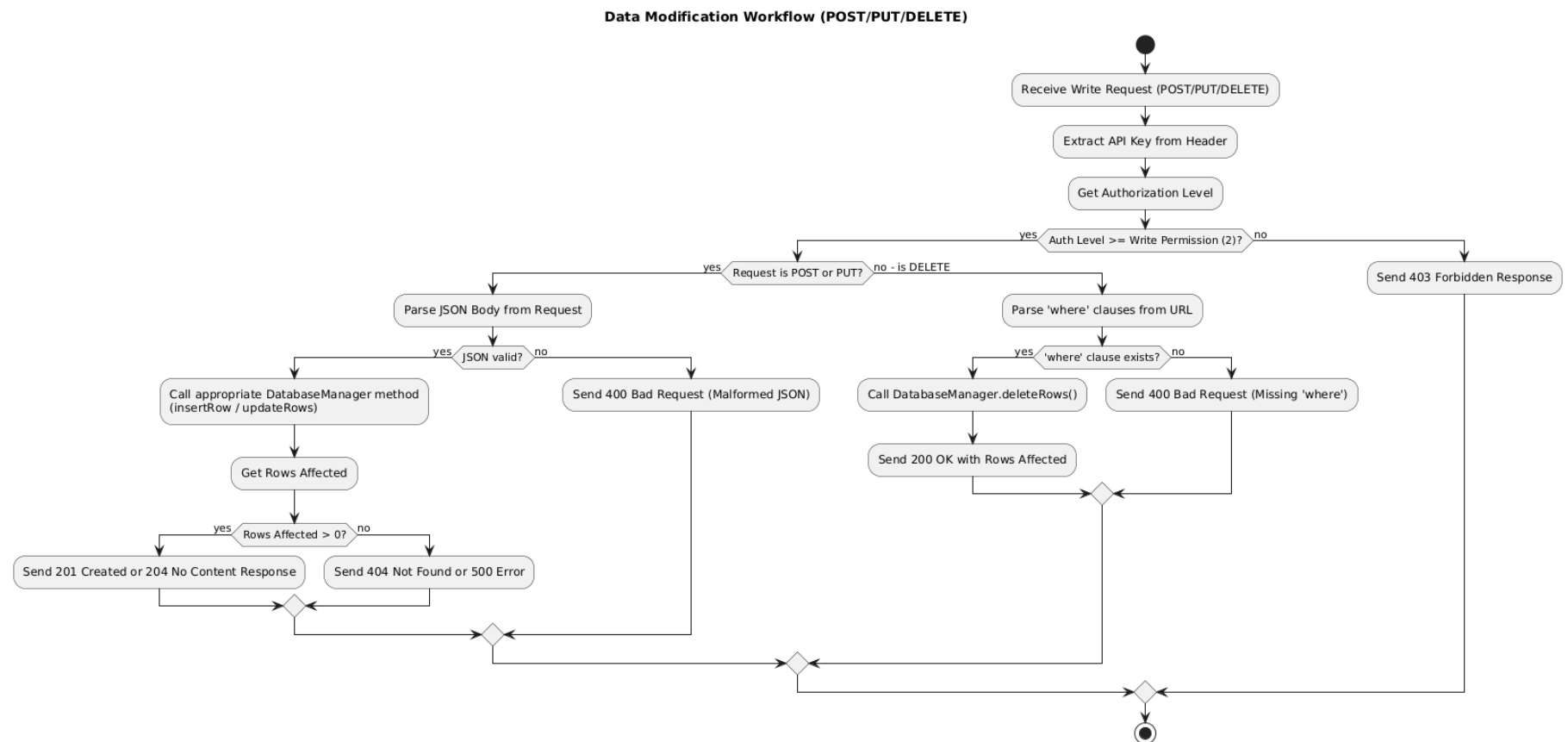


Design: Activity Diagram (Data Request)



Design: Activity Diagram (Data Modification)

- This flowchart details the process for handling write operations like POST, PUT, and DELETE.



Design: OOP Principles in Action

- **Encapsulation:** The `DatabaseManager` hides all complex database logic and exposes only a clean, simple interface.
- **Inheritance:** All request handlers **extend** a common `BaseRequestHandler` to inherit and reuse critical code.
- **Polymorphism (Strategy Pattern):** The `BaseExporter` and its subclasses (`JsonExporter`, `CsvExporter`) allow for flexible, runtime selection of the data format.
- **Abstraction:** The entire API is an **abstraction layer** over the database. The client is shielded from all underlying complexity.

Implementation: Server Startup

- This screenshot shows the live console output when the server starts.

```
data-integration-api on  main [?] via  via  v21.0.6
> ./start.sh

🚀 API Server is running!



🏠 Local Access:



http://localhost:8080/api/v1  
http://127.0.0.1:8080/api/v1



🌐 Network Access:



http://192.168.1.9:8080/api/v1



💡 Press Ctrl+C to stop the server
```

Implementation: Server Logging

- This screenshot shows the live console output and real-time request logging
- helpful for later debugging and error resolving.

```
2025-10-05 20:03:56.847 INFO >>> method=GET uri="/api/v1/data/tables" status=404
2025-10-05 20:04:01.029 INFO >>> method=GET uri="/api/v1/data/" status=400
2025-10-05 20:04:04.416 INFO >>> method=GET uri="/api/v1/data/products" status=200
2025-10-05 20:04:19.872 INFO >>> method=GET uri="/api/v1/data/products" status=200
2025-10-05 20:04:20.357 INFO >>> method=GET uri="/api/v1/data/products" status=200
2025-10-05 20:04:20.573 INFO >>> method=GET uri="/api/v1/data/products" status=200
2025-10-05 20:04:20.755 INFO >>> method=GET uri="/api/v1/data/products" status=200
2025-10-05 20:04:20.915 INFO >>> method=GET uri="/api/v1/data/products" status=200
2025-10-05 20:04:21.056 INFO >>> method=GET uri="/api/v1/data/products" status=200
2025-10-05 20:04:21.261 INFO >>> method=GET uri="/api/v1/data/products" status=200
2025-10-05 20:04:32.426 INFO >>> method=GET uri="/api/v1/data/products?where:id=4" status=200
2025-10-05 20:04:37.819 INFO >>> method=GET uri="/api/v1/data/products?where:no=4" status=200
2025-10-05 20:04:47.424 INFO >>> method=GET uri="/api/v1/data/products?where:id=4" status=400
2025-10-05 20:05:03.158 INFO >>> method=GET uri="/api/v1/data/products?where:id:eq:4" status=200
2025-10-05 20:05:10.605 INFO >>> method=GET uri="/api/v1/data/products/5" status=200
```

Implementation: POST Request

- Here, a **PUT** request is sent via **curl** to update a product detail, demonstrating the write capabilities of the API.

```
~
> curl http://127.0.0.1:8080/api/v1/data/products/1?fields=name,price&format=md|glow
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left     Speed
100    60    100    60      0      0  17730      0  --:--:--  --:--:--  --:--:--  20000

name      | price
-----|-----
Rustic System | 2399.0

~
> curl -X PUT \
  -H "Authorization: Bearer admin-safe-key" \
  -H "Content-Type: application/json" \
  -d '{"price": 7000.00}' \
  "http://localhost:8080/api/v1/data/products/1"

~
> curl http://127.0.0.1:8080/api/v1/data/products/1?fields=name,price&format=md|glow
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left     Speed
100    60    100    60      0      0  24559      0  --:--:--  --:--:--  --:--:--  30000

name      | price
-----|-----
Rustic System | 7000.0
```

Implementation: JSON Output

- This shows the default data output format. The API returns a clean, well-structured JSON array.

```
> curl http://127.0.0.1:8080/api/v1/data/products?where=price:gt:900&where=stock:lt:800&replace=name:items&sort=price:desc&format=json|jq
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           % Done      0         0             0          0             0          0
100  1067  100  1067    0     0  285k      0 --:--:-- --:--:-- --:--:-- 347k
[
  {
    "id": 76,
    "items": "Ergonomic Unit",
    "category": "Toys",
    "price": 994.78,
    "stock": 733
  },
  {
    "id": 93,
    "items": "Ergonomic Gadget",
    "category": "Office",
    "price": 987.1,
    "stock": 12
  },
  {
    "id": 38,
    "items": "Digital Appliance",
    "category": "Outdoors",
    "price": 985.33,
    "stock": 756
  },
  {
    "id": 75,
    "items": "Modern System",
    "category": "Toys",
    "price": 980.26,
    "stock": 424
  },
  {
    "id": 78,
    "items": "Modern Unit",
    "category": "Outdoors",
    "price": 973.11,
    "stock": 568
  },
]
```

Implementation: CSV Output

- By simply adding `?format=csv` to the URL, the same data is returned in CSV format, ready for spreadsheets or data analysis tools.

```
~  
> curl http://127.0.0.1:8080/api/v1/data/products?where=price:gt:900&where=stock:lt:800&replace=name:items&sort=price:desc&format=csv  
id,items,category,price,stock  
76,Ergonomic Unit,Toys,994.78,733  
93,Ergonomic Gadget,Office,987.1,12  
38,Digital Appliance,Outdoors,985.33,756  
75,Modern System,Toys,980.26,424  
78,Modern Unit,Outdoors,973.11,568  
92,Digital Appliance,Toys,943.2,652  
58,Vintage Unit,Industrial,933.31,590  
47,Compact Appliance,Electronics,933.13,328  
  
~  
> |
```

Implementation: HTML Output

- Using `?format=html`, the API generates a user-friendly, formatted HTML table for easy viewing in a web browser.

Data Export

id	items	category	price	stock
76	Ergonomic Unit	Toys	994.78	733
93	Ergonomic Gadget	Office	987.1	12
38	Digital Appliance	Outdoors	985.33	756
75	Modern System	Toys	980.26	424
78	Modern Unit	Outdoors	973.11	568
92	Digital Appliance	Toys	943.2	652
58	Vintage Unit	Industrial	933.31	590
47	Compact Appliance	Electronics	933.13	328

Implementation: Markdown Output

- The API can even generate a Markdown table with `?format=md`, useful for documentation or notes.

```
~
> curl http://127.0.0.1:8080/api/v1/data/products?where=price:gt:900&where=stock:lt:800&replace=name:items&sort=price:desc&format=md |
  glow
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left  Speed
100   472   100   472    0    0   106k      0 --:--:-- --:--:-- --:--:--  115k

id | items | category | price | stock
---|---|---|---|---
76 | Ergonomic Unit | Toys | 994.78 | 733
93 | Ergonomic Gadget | Office | 987.1 | 12
38 | Digital Appliance | Outdoors | 985.33 | 756
75 | Modern System | Toys | 980.26 | 424
78 | Modern Unit | Outdoors | 973.11 | 568
92 | Digital Appliance | Toys | 943.2 | 652
58 | Vintage Unit | Industrial | 933.31 | 590
47 | Compact Appliance | Electronics | 933.13 | 328

~
> |
```

Result: Demonstrated Outputs

- created a **fully functional, standalone RESTful API server**.
- The server is packaged as a single, runnable JAR file with no external frameworks required—it only needs a Java Virtual Machine.

Result: How Objectives Were Met

- **Decoupling:** Achieved via the abstraction layer and view management.
- **SQL Abstraction:** Achieved via the URL-based dynamic query engine.
- **Transformation:** Achieved via the polymorphic `Exporter` module.
- **Security:** Achieved via the API key and RBAC implementation.

Result: Comparison with Expected Results

- Our goal was to create a working prototype that proved the core concepts.
- The final result exceeded expectations, delivering a stable, light-weight and highly functional application that is ready for initial use.
- Implemented RBAC which wasn't in the initial plan.
- well abstracted code with well defined modules.

Future Scope: Features

- **Enhanced Querying:** Add support for `JOIN` operations between tables through the API.
- **Logical Operators:** Implement support for complex `OR` conditions in `where` clauses.

Future Scope: Improvements

- **Performance:** Introduce a caching layer (e.g., using Caffeine or Ehcache) to store results of frequent queries, reducing database load.
- **User Experience:** Develop a simple GUI Admin Panel (web interface) for managing API keys, views, and browsing data.
- **Key Revoke** Implement revoking key incase of security breach.

Future Scope: Scalability

- **Database Support:** Add official support and testing for enterprise-grade databases like PostgreSQL and MySQL.
- **Deployment:** Create a **Docker image** for easy, containerized deployment in any environment.

Conclusion: Summary of Achievement

- We successfully designed and built a lightweight, powerful, and flexible data gateway that solves the critical problem of tight coupling in modern software.
- The project is a case study in applying core OOP principles to build an elegant and maintainable solution without relying on heavy frameworks like spring.
- Achieved all of the core functionality with little to no dependancy of any external packages.

Conclusion: Importance in Real-World Applications

- This tool dramatically **reduces development time and costs** by eliminating repetitive client-side code.
- It improves **system stability and resilience** by decoupling components.
- It enhances **security** by providing a centralized and managed access point to data.

Conclusion: Overall Learning Experience

- This project provided deep, practical experience in applying OOP principles (Encapsulation, Inheritance, Polymorphism, Abstraction) to build a real-world application from the ground up.
- It reinforced the importance of good architectural design and the long-term benefits of creating a modular, loosely coupled system.

Q & A

THANK YOU



Made with Markweavia

Markdown, beautifully woven.

Create your own Slides